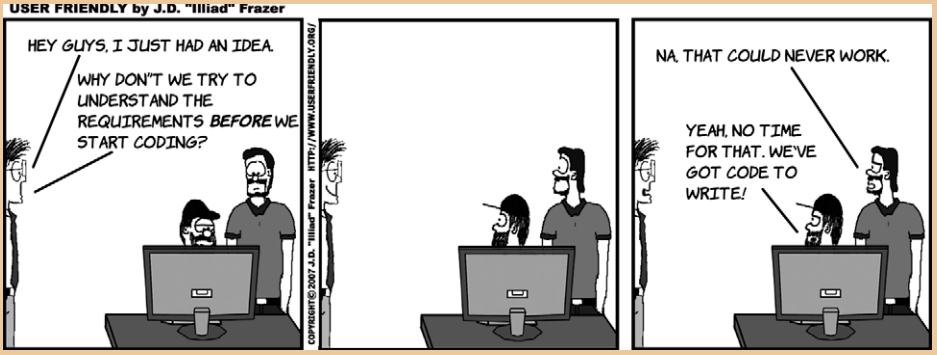
What is Test-Driven Development?

Test-Driven Development (TDD) is a development practice that uses unit tests to specify, design, and verify the code you're writing. Before implementing a piece of functionality, developers write a failing unit test that demonstrates how this functionality should work. At the same time, this failing test also proves that the current implementation doesn't yet support the new functionality. Only then do the developers write the application code. Once the unit test passes, the developers know that the functionality has been successfully implemented. At this stage, they can review their code to tidy things up and fine-tune the design.

TDD has been around for a long time and can be traced back to Extreme Programming practices developed in the late 1990s. TDD relies on two simple principles:

- Don't write any code until you've written a failing test that demonstrates why you need this code.
- Refactor regularly to avoid duplication and keep the code quality high.

The cornerstone of TDD is the idea of writing a unit test before you write the corresponding code. But TDD is much more than just a guarantee that every class has a corresponding set of unit tests. With enough discipline, any experienced developer writing unit tests after writing the code can achieve that outcome as well. TDD's killer feature is that it forces developers to think about the code they're going to write before they write it, in practical and unambiguous terms—you need to understand the functionality before you can write a unit test for it. This way, developers resist the temptation to just start coding something and actively think about what they need to achieve. In this respect, a better term for this practice might be something like "Test-Driven Design."



TDD's killer feature is that it forces developers to think about the requirements before they start to code. This approach isn't always easy for teams to adopt.

This somewhat counterintuitive idea arguably represents one of the single most significant contributions to software engineering quality that has happened over the last 20 years. TDD offers several benefits:

- Cleaner, better designed code
- Code that's easier and less expensive to maintain